Полная схема: от запроса пользователя до ответа модели

```
╔═══════════════════════════════════════════════════════════════════╗
║  1. ВВОД ПОЛЬЗОВАТЕЛЯ                                              ║
║     "Расскажи анекдот про кота"                                   ║
╚═══════════════════╤═══════════════════════════════════════════════╝
                    │
                    ▼

╔═══════════════════════════════════════════════════════════════════╗
║  2. ТОКЕНИЗАЦИЯ                                                    ║
║                                                                   ║
║  Текст разбивается на токены по словарю модели (BPE алгоритм)     ║
║                                                                   ║
║  "Расскажи анекдот про кота"                                      ║
║       ↓                                                           ║
║  ["Рас", "скажи", " анек", "дот", " про", " кота"]                ║
║                                                                   ║
║  Каждый токен получает числовой ID из словаря:                    ║
║  [4821, 9903, 7712, 3304, 1082, 8845]                             ║
╚═══════════════════╤═══════════════════════════════════════════════╝
                    │
                    ▼

╔═══════════════════════════════════════════════════════════════════╗
║  3. ЭМБЕДДИНГ (Embedding)                                          ║
║                                                                   ║
║  Каждый token ID превращается в вектор — массив чисел,             ║
║  который кодирует СМЫСЛ токена в многомерном пространстве         ║
║                                                                   ║
║  ID 4821 → [0.12, -0.45, 0.78, 0.03, ..., -0.21]  (2048+ чисел)║
║  ID 9903 → [0.56, 0.11, -0.33, 0.89, ..., 0.44]                  ║
║  ...                                                              ║
║                                                                   ║
║  + Positional Encoding — добавляется информация о ПОЗИЦИИ         ║
║    каждого токена в предложении (модель должна знать порядок)     ║
╚═══════════════════╤═══════════════════════════════════════════════╝
                    │
                    ▼

╔═══════════════════════════════════════════════════════════════════╗
║  4. ТРАНСФОРМЕР (Transformer)                                      ║
║                                                                   ║
║  Ядро модели. Набор слоёв (у Gemini — десятки слоёв),             ║
║  каждый слой содержит:                                            ║
║                                                                   ║
║  ┌──────────────────────────────────────────────────────────┐    ║
║  │  Self-Attention (механизм внимания)                      │    ║
║  │                                                          │    ║
```

```
│  │  Каждый токен "смотрит" на все остальные токены          │    │
│  │  и определяет, какие из них важны для текущего контекста  │    │
│  │                                                          │    │
│  │  "кота" обращает внимание на "анекдот" и "расскажи"      │    │
│  │  → понимает: нужен юмор про кота                         │    │
│  └──────────────────────────────────────────────────────────┘    │
│                          │                                       │
│                          ▼                                       │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │  Feed-Forward Network (нейросеть прямого прохода)        │    │
│  │                                                          │    │
│  │  Обрабатывает результат attention, извлекает              │    │
│  │  более сложные закономерности и "знания"                 │    │
│  └──────────────────────────────────────────────────────────┘    │
│                                                                  │
│  Это повторяется N раз (слой за слоем), каждый раз углубляя     │
│  понимание контекста                                             │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼

┌──────────────────────────────────────────────────────────────────┐
│  5. ЛОГИТЫ (Logits)                                              │
│                                                                  │
│  На выходе трансформера — один вектор, который проецируется      │
│  на ВЕСЬ словарь. Получаем сырую оценку для каждого токена:      │
│                                                                  │
│  "Идёт"    →  5.8                                                │
│  "Жил"     →  5.2                                                │
│  "Кот"     →  4.1                                                │
│  "Однажды" →  3.9                                                │
│  "The"     → -2.1                                                │
│  "xyz"     → -9.4                                                │
│  ... (30 000+ оценок — по одной на каждый токен в словаре)       │
│                                                                  │
│  Это ещё НЕ вероятности, просто числа                            │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼

┌──────────────────────────────────────────────────────────────────┐
│  6. СЭМПЛИРОВАНИЕ (выбор следующего токена)                      │
│                                                                  │
│  6.1  Temperature                                                │
│  ─────────────────                                               │
│  Каждый логит делится на temperature:                            │
│  logit / temperature                                             │
│                                                                  │
```

```
│  Низкая (0.1) → разница усиливается → предсказуемый ответ        │
│  Высокая (2.0) → разница сглаживается → креативный ответ         │
│                          │                                       │
│                          ▼                                       │
│  6.2  Top-K                                                      │
│  ─────────────────                                               │
│  Оставляем только K самых вероятных токенов                      │
│  (topK=64 → берём 64, остальные 29 936 отбрасываем)              │
│                          │                                       │
│                          ▼                                       │
│  6.3  Top-P (Nucleus Sampling)                                   │
│  ─────────────────                                               │
│  Из оставшихся берём минимальный набор, чья суммарная            │
│  вероятность ≥ 95% (topP=0.95)                                   │
│                          │                                       │
│                          ▼                                       │
│  6.4  Softmax → Вероятности                                      │
│  ─────────────────                                               │
│  Преобразуем отфильтрованные логиты в вероятности:               │
│                                                                  │
│  "Идёт"    → 42%                                                 │
│  "Жил"     → 31%                                                 │
│  "Кот"     → 18%                                                 │
│  "Однажды" →  9%                                                 │
│                          │                                       │
│                          ▼                                       │
│  6.5  Случайный выбор (sampling)                                 │
│  ─────────────────                                               │
│  Бросаем "кубик" с учётом вероятностей                           │
│  Результат: "Идёт"                                               │
└──────────────────────────┬──────────────────────────────────────┘
                           │
                           ▼

┌──────────────────────────────────────────────────────────────────┐
│  7. ПРОВЕРКА ОСТАНОВКИ                                           │
│                                                                  │
│  Сгенерированный токен = EOS (конец)?     → СТОП                 │
│  Достигнут maxOutputTokens?                → СТОП                │
│  Встретился stop sequence?                 → СТОП                │
│  Ничего из этого?                          → ПРОДОЛЖАЕМ ↓        │
└──────────────────────────┬──────────────────────────────────────┘
                           │
                           ▼

┌──────────────────────────────────────────────────────────────────┐
│  8. АВТОРЕГРЕССИОННЫЙ ЦИКЛ                                       │
│                                                                  │
```

```
╟                                                                    ╟
╟  Выбранный токен добавляется к входной последовательности:        ╟
╟                                                                    ╟
╟  Итерация 1: "Расскажи анекдот про кота"          → "Идёт"      ╟
╟  Итерация 2: "Расскажи анекдот про кота Идёт"     → "кот"       ╟
╟  Итерация 3: "Расскажи анекдот про кота Идёт кот" → "по"        ╟
╟  ...                                                               ╟
╟                                                                    ╟
╟  Каждая итерация проходит ЗАНОВО шаги 3 → 4 → 5 → 6 → 7        ╟
╟  (с учётом KV-cache для ускорения)                                 ╟
╚═══════════════════════════╤═══════════════════════════════════════╝
                            │
                            ▼

╔═══════════════════════════════════════════════════════════════════╗
╟  9. ДЕТОКЕНИЗАЦИЯ                                                 ╟
╟                                                                    ╟
╟  Набранные токены склеиваются обратно в текст:                    ╟
╟                                                                    ╟
╟  ["Идёт", " кот", " по", " крыше", "..."]                        ╟
╟      ↓                                                            ╟
╟  "Идёт кот по крыше..."                                           ╟
╟                                                                    ╟
╟  Этот текст отдаётся пользователю                                 ╟
╟  (при streaming — токен за токеном по мере генерации)             ╟
╚═══════════════════════════════════════════════════════════════════╝
```

Разбор параметров модели Google Gemini

Идентификация модели

Параметр	Значение в примере	Описание
`name`	`"models/gemini-2.5-flash"`	Уникальный идентификатор модели для использования в API-запросах. Это то, что передаётся в поле `model` при вызове генерации
`version`	`"001"`	Версия модели. Google выпускает обновления одной и той же модели — `001`, `002` и т.д.
`displayName`	`"Gemini 2.5 Flash"`	Человекочитаемое название для UI/документации
`description`	`"Stable version of..."`	Текстовое описание: что за модель, когда выпущена, ключевые возможности

Лимиты токенов

Параметр	Значение	Описание
		Максимальный размер **входного** контекста в токенах. 1M

`inputTokenLimit`	`1048576` (1M)	токенов ≈ ~700-800 тыс. слов — можно отправить целую книгу или огромную кодовую базу
`outputTokenLimit`	`65536` (64K)	Максимальный размер **ответа** модели в токенах. Модель не сможет сгенерировать ответ длиннее этого лимита

Поддерживаемые методы API

`supportedGenerationMethods` — массив доступных API-эндпоинтов для этой модели:

Метод	Назначение
`generateContent`	Основной метод — отправить промпт, получить ответ (аналог `/chat/completions` у OpenAI)
`countTokens`	Подсчёт токенов в тексте **без** генерации. Полезно для проверки, влезет ли текст в контекстное окно
`createCachedContent`	Кэширование контекста — можно сохранить большой текст (напр. документацию) и переиспользовать в нескольких запросах, экономя деньги и время
`batchGenerateContent`	Пакетная генерация — отправка множества запросов одним вызовом. Дешевле, но ответ приходит не сразу

> *Если модель **не** поддерживает метод (например, `batchGenerateContent` отсутствует) — вызов этого эндпоинта вернёт ошибку.*

Параметры сэмплирования (настройки "креативности")

Эти параметры управляют тем, **насколько случайным/креативным** будет ответ:

Параметр	Значение	Описание
`temperature`	`1`	**Значение по умолчанию.** Контролирует "случайность" выбора следующего токена. `0` = детерминированный (всегда самый вероятный токен), `2` = максимально креативный/рандомный. `1` — баланс
`maxTemperature`	`2`	Максимально допустимое значение temperature для этой модели. Если передать больше — API вернёт ошибку
`topP`	`0.95`	**Nucleus sampling.** Модель выбирает следующий токен только из множества самых вероятных, чья суммарная вероятность = 95%. Отсекает маловероятные "мусорные" токены
`topK`	`64`	Модель рассматривает только **64 самых вероятных** токена на каждом шаге. Работает совместно с `topP`

Как temperature, topP и topK работают вместе:

```
Все возможные токены (30,000+)
        │
        ▼
   ┌─ topK = 64 ──────────────────┐
   │ Берём только 64 самых        │
```

```
│  берем только 64 самых          │
│  вероятных токена               │
└─────────────┬─────────────────────┘
              │
              ▼
┌─ topP = 0.95 ─────────────────┐
│  Из них оставляем те, чья     │
│  суммарная вероятность ≤ 95%  │
└─────────────┬─────────────────────┘
              │
              ▼
┌─ temperature = 1.0 ──────────┐
│  Применяем temperature к      │
│  оставшимся: чем выше —       │
│  тем равномернее распределение │
└─────────────┬─────────────────────┘
              │
              ▼
        Выбранный токен
```

Thinking (режим рассуждений)

Параметр	Значение	Описание
`thinking`	`true`	Модель поддерживает **thinking mode** — "цепочку рассуждений" (Chain of Thought). Модель сначала "думает" (пишет внутренние рассуждения), а потом формирует финальный ответ. Улучшает качество на сложных задачах (математика, код, логика). Аналог "thinking" у Claude

---

Резюме

По сути, этот объект — **карточка возможностей модели**. Клиент может программно:

- выбрать модель с нужным размером контекста
- проверить, поддерживает ли модель нужный метод
- установить корректные дефолты для temperature/topP/topK
- не выходить за `maxTemperature`

Как работает topP (Nucleus Sampling)

topP = 0.95 означает:

1. Токены сортируются по убыванию вероятности
2. Мы **накапливаем** их вероятности сверху вниз
3. Останавливаемся, когда **сумма набрала 95%**

```
Токен A:  40%  → сумма: 40%   ✅ берём
Токен B:  25%  → сумма: 65%   ✅ берём
Токен C:  18%  → сумма: 83%   ✅ берём
Токен D:  10%  → сумма: 93%   ✅ берём
Токен E:   4%  → сумма: 97%   ✅ берём (перешли порог — но этот токен включается)
Токен F:   2%  → сумма: 99%   ❌ отсекаем
```

```
Токен F:   2%  → сумма: 99%  ❌ отсекаем
Токен G:   1%  → сумма: 100% ❌ отсекаем
```

Мы берём **минимальное множество токенов**, чья **суммарная вероятность ≥ 95%**, и выбираем из них. Все остальные (маловероятные "хвостовые" токены) — отсекаются.

Так что вы правы — корректнее было бы написать **≥ 95%**, а не ≤ 95%. Суммарная вероятность отобранных токенов должна быть **не меньше** порога.

Хотите, чтобы я обновил PDF с исправленным знаком?

Как работает temperature

Temperature применяется к логитам (сырым оценкам) модели **до** преобразования в вероятности через softmax:

```
adjusted_logit = logit / temperature
```

temperature = 0.1 (низкая)

```
Токен A: 90%  ▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬
Токен B:  8%  ▬▬
Токен C:  2%  |
```

Распределение **острое** — модель почти всегда выберет токен A. Результат **детерминированный**, предсказуемый.

temperature = 1.0 (стандартная)

```
Токен A: 50%  ▬▬▬▬▬▬▬▬▬▬
Токен B: 30%  ▬▬▬▬▬▬
Токен C: 20%  ▬▬▬▬
```

Оригинальное распределение модели.

temperature = 2.0 (высокая)

```
Токен A: 38%  ▬▬▬▬▬▬▬▬
Токен B: 33%  ▬▬▬▬▬▬▬
Токен C: 29%  ▬▬▬▬▬▬
```

Распределение **сглаженное, равномерное** — все токены получают более похожие вероятности. Результат **случайный**, креативный.

Логика

- **Делим на большое число** → различия между логитами сжимаются → вероятности выравниваются → **равномернее**
- **Делим на маленькое число** → различия усиливаются → лидер доминирует → **острее**

Так что **чем выше temperature — тем равномернее** распределение. Формулировка в PDF верная.

Объясняю пошагово

Шаг 1: Модель генерирует "логиты"

Когда модель выбирает следующее слово, она внутри себя присваивает **числовую оценку (score)** каждому возможному токену. Эти сырые оценки называются **логиты**.

```
"привет"  → 5.0
"здравствуй" → 3.0
"пока"    → 1.0
```

Это ещё **не вероятности** — это просто числа, которые показывают, насколько модель "уверена" в каждом варианте.

Шаг 2: Применяем temperature

Каждый логит **делится** на значение temperature:

```
temperature = 2.0

"привет"     → 5.0 / 2.0 = 2.5
"здравствуй" → 3.0 / 2.0 = 1.5
"пока"       → 1.0 / 2.0 = 0.5
```

Видите? Разница между числами **сократилась**. Было 5.0 vs 1.0 (разница 4), стало 2.5 vs 0.5 (разница 2).

Шаг 3: Softmax превращает числа в вероятности

Softmax — это математическая функция, которая берёт любые числа и превращает их в вероятности (от 0 до 1, в сумме = 100%):

Без temperature (делим на 1):

```
логиты: 5.0, 3.0, 1.0
              ↓ softmax
"привет"     → 84%
"здравствуй" → 14%
"пока"       →  2%
```

"Привет" доминирует — почти всегда будет выбран.

С temperature = 2.0 (делим на 2):

```
логиты: 2.5, 1.5, 0.5
              ↓ softmax
"привет"     → 53%
"здравствуй" → 30%
"пока"       → 17%
```

Вероятности стали **ближе друг к другу** — у модели больше "свободы" выбрать разные варианты.

Итого весь процесс:

```
Логиты (сырые оценки)
        |
        ↓  делим на temperature
Скорректированные логиты
        |
        ↓  softmax
Вероятности (0-100%)
        |
        ↓  случайный выбор
```

```
Выбранный токен
```

Вот что значит фраза "temperature применяется к логитам **до** softmax" — сначала корректируем числа, потом превращаем в вероятности.

а что происходит после того как мы выбрали токен?

Что происходит после выбора токена

Процесс повторяется **в цикле** — токен за токеном, пока ответ не будет готов:

```
Шаг 1:  Вход: "Расскажи анекдот"
        Модель выбирает токен → "Идёт"

Шаг 2:  Вход: "Расскажи анекдот Идёт"
        Модель выбирает токен → "медведь"

Шаг 3:  Вход: "Расскажи анекдот Идёт медведь"
        Модель выбирает токен → "по"

Шаг 4:  Вход: "Расскажи анекдот Идёт медведь по"
        Модель выбирает токен → "лесу"

...и так далее
```

На **каждом** шаге заново проходит весь pipeline:

```
┌──────────────────────────────────────────────┐
│  Весь предыдущий текст + уже сгенерированные │
│  токены подаются на вход модели              │
│                    ↓                         │
│  Модель считает логиты для ВСЕХ токенов      │
│                    ↓                         │
│  topK → topP → temperature → softmax         │
│                    ↓                         │
│  Выбирается ОДИН следующий токен             │
│                    ↓                         │
│  Токен добавляется к ответу                  │
│                    ↓                         │
│  Повторяем ──────────────────────────┐       │
└──────────────────────────────────────┘       │
       ↑                                       │
       └───────────────────────────────────────┘
```

Когда цикл останавливается?

Генерация прекращается при одном из условий:

Условие	Пример
Модель сгенерировала стоп-токен	Специальный токен `<EOS>` (End of Sequence) — модель сама решила, что ответ завершён

Достигнут outputTokenLimit	Для Gemini 2.5 Flash это 65 536 токенов — жёсткий потолок
Достигнут ваш maxOutputTokens	Вы можете в запросе установить свой лимит, например 1000 токенов
Сработал stop sequence	Можно задать стоп-слово в параметрах запроса — например, `"stop": ["###"]`

Важный момент

Модель **не знает заранее**, что она напишет. У неё нет "плана" всего ответа. Каждый токен — это отдельное решение на основе всего предыдущего контекста. Именно поэтому это называется **авторегрессионная генерация** — каждый следующий шаг зависит от всех предыдущих.

Токен — это кусочек текста

Модель работает не с буквами и не с целыми словами, а с **токенами** — фрагментами текста:

```
"Привет, как дела?"

разбивается на токены:
["Прив", "ет", ",", " как", " дела", "?"]
```

У модели есть **словарь** всех возможных токенов (обычно 30 000 — 100 000 штук). Каждый токен — это просто элемент этого словаря.

Логит — это оценка для каждого токена

Когда модель решает, какой токен поставить следующим, она выставляет **числовую оценку** каждому токену из словаря. Эти оценки и есть логиты:

```
Словарь: 30 000 токенов
Модель выставляет 30 000 логитов — по одному на каждый:

Токен "кот"    → логит  5.2
Токен "собака" → логит  3.1
Токен "стул"   → логит -1.4
Токен "xyz"    → логит -8.7
...и так для каждого токена в словаре
```

Аналогия

Ответ на вопрос

Логиты присваиваются ДО topK и topP. Порядок такой:

1. Трансформер выдаёт логиты для всех 30 000+ токенов
2. Temperature корректирует логиты
3. Top-K отсекает по количеству
4. Top-P отсекает по суммарной вероятности
5. Softmax переводит в вероятности
6. Sampling выбирает один токен

Логиты — это результат работы нейросети. topK/topP — это уже фильтрация после неё.